Parser

terça-feira, 22 de julho de 2025 07:56

↳ Representing the code
↳ tree that represents the grammatical structure
↳ Context free grammar (CFG)
↳ Instead of create infinite rules → create just what we can for our CLASS

Rules → Productions
↳ each production has a head & Body
describes what it generates

Terminals
↳ tokens meaning end-points
↳ nothing derives from them → produce 1 symbol

In each step of the game, you pick a **rule** and follow what it tells you to do. Most of the lingo around formal grammars comes from playing them in this direction. **Rules** are called **productions** because they produce strings in the grammar.

Each **production** in a context-free grammar has a **head**—its name—and a **body** which describes what it generates. In its pure form, the body is simply a list of **symbols**. Symbols come in two delectable flavors:

A **terminal** is a letter from the grammar's alphabet. You can think of it like a literal value. In the syntactic grammar we're defining, the terminals are individual lexemes—tokens coming from the scanner like if or 1234. These are called "terminals", in the sense of an "end point" because they don't lead to any further "moves" in the game. You simply produce that one symbol.

A **nonterminal** is a named reference to another rule in the grammar. It means "play that rule and insert whatever it produces here". In this way, the grammar composes.

I tried to come up with something clean. Each rule is a name, followed by an arrow (→), followed by a sequence of symbols, and finally ending with a semicolon (;). Terminals are quoted strings, and nonterminals are lowercase words.

```
breakfast → protein "with" breakfast "on the side" ;
breakfast → protein ;
breakfast → bread ;
protein → crispiness "crispy" "bacon" ;
protein → "sausage" ;
protein → cooked "eggs" ;
crispiness → "really" ;
crispiness → "really" crispiness ;
cooked → "scrambled" ;
cooked → "poached" ;
cooked → "fried" ;
bread → "toast" ;
bread → "biscuits" ;
bread → "English muffin" ;
```

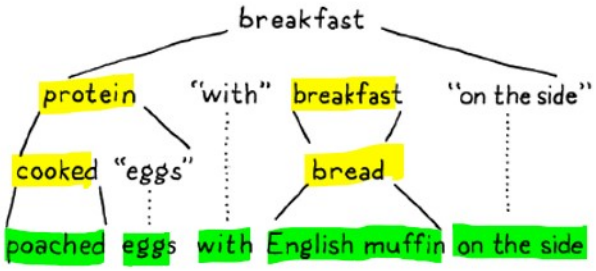
→ Expand the non-terminal w/ the possibilities to create a new sentence (expression w/ variables)
↳ there is no context in this case
→ Every non terminal can be expanded until it contains only terminals

With that, every nonterminal in the string has been expanded until it finally contains only terminals and we're left with:

only terminals

With that, every nonterminal in the string has been expanded until it finally contains only terminals and we're left with:

→ Mean it possible to have short number of grammars & encode a large set of strings



- Symbols → Represent entire tokens
- Rules →
- Productions →

→ I may need to declare this grammar rule for each type

Expressions

- Literals – Numbers, strings, Booleans, and nil.
- Unary expressions – A prefix ! to perform a logical not, and - to negate a number.
- Binary expressions – The infix arithmetic (+, -, *, /) and logic (==, !=, <, <=, >, >=) operators we know and love.
- Parentheses – A pair of (and) wrapped around an expression.

In addition to quoted strings for **terminals that match exact lexeme**s, we CAPITALIZE terminals that are a single lexeme whose text representation may vary. NUMBER is any number literal, and STRING is any string literal. Later, we'll do the same for IDENTIFIER.

→ Terminals can match lexemes !!!

Using our handy dandy new notation, here's a grammar for those:

```
expression → literal
           | unary
           | binary
           | grouping ;
literal → NUMBER | STRING | "true" | "false" | "nil" ;
grouping → "(" expression ")" ;
unary → ( "-" | "!" ) expression ;
binary → expression operator expression ;
operator → "==" | "!=" | "<" | "<=" | ">" | ">="
         | "+" | "-" | "*" | "/" ;
```